How to architect an AI-first company

Start with business outcomes. Connect the work. Build systems that can improve.

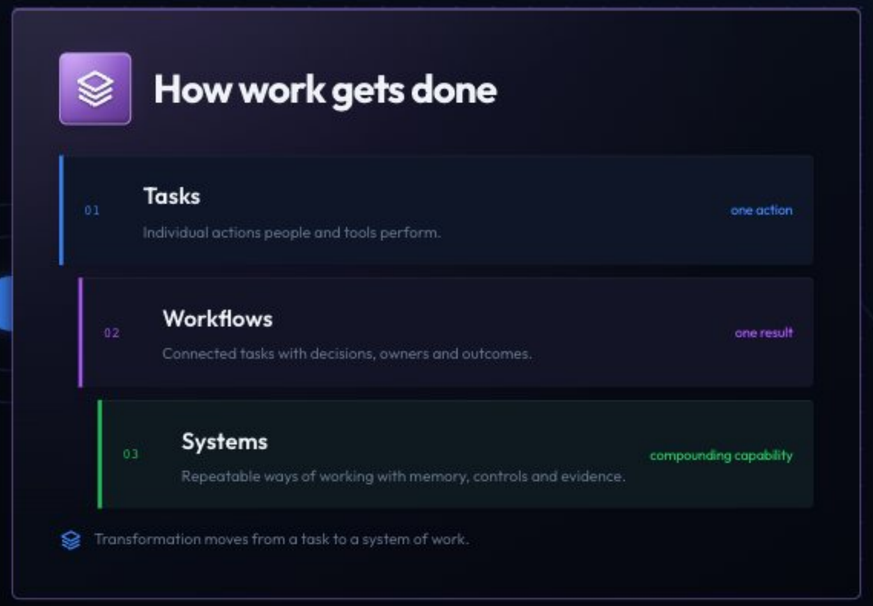
Start with the business objective.

Use AI to create more value: increase capacity, improve quality and lower the cost of getting work done.



Ask how work gets done.

Tasks are individual actions. Workflows connect them. Systems make the whole way of working repeatable.



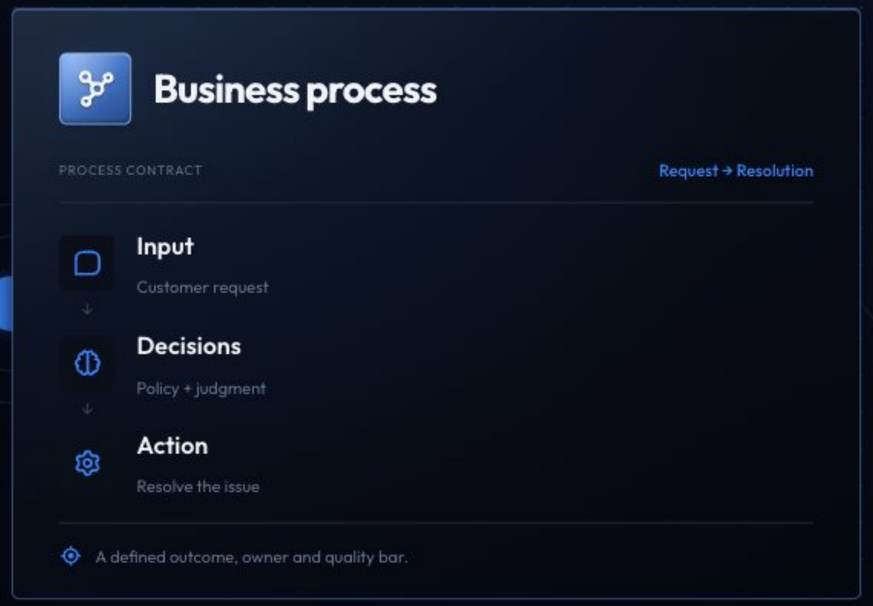
Choose one *valuable workflow.*

Start with a recurring business problem, redesign the work and prove the result before expanding.



A workflow delivers a *business result.*

Define the input, decisions, actions and outcome before choosing the technology.



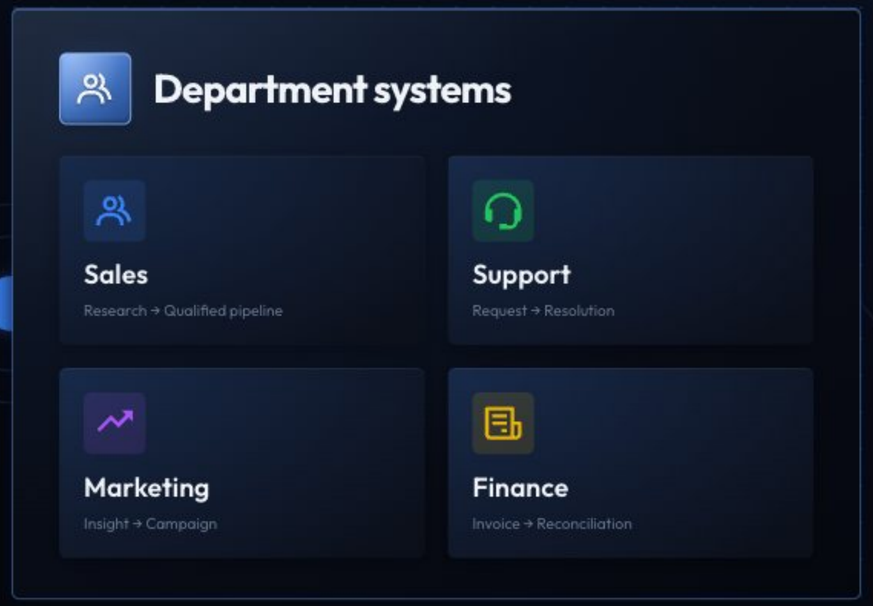
One win becomes a *team capability.*

Combine proven workflows to change how a department handles an entire class of work.



Share the foundation.
Adapt the work.

Departments use common infrastructure while keeping their own context, workflows and success measures.



From a business goal to a company capability.

Connect the tasks into a workflow. Make the workflow dependable as a system. Reuse proven systems across departments.

01 DEFINE THE OUTCOME

Business goal

Resolve faster.

- Time to resolution
- Cost per resolution
- Quality floor

02 UNDERSTAND THE WORK

Tasks

Make the work visible.

01 Read the request
02 Find order + policy
03 Prepare a reply

03 CONNECT THE STEPS

Workflow

One end-to-end result.

- Ticket → context
- Draft → review gate
- Reply or escalate

04 MAKE IT DEPENDABLE

System

Run it with control.

01 Context + tools
02 Owner + guardrails
03 Traces + evals

05 REUSE WHAT WORKS

Department

A repeatable capability.

01 Delivery exceptions
02 Returns + refunds
03 More proven workflows

Measure the result. Improve the system. Expand what works.

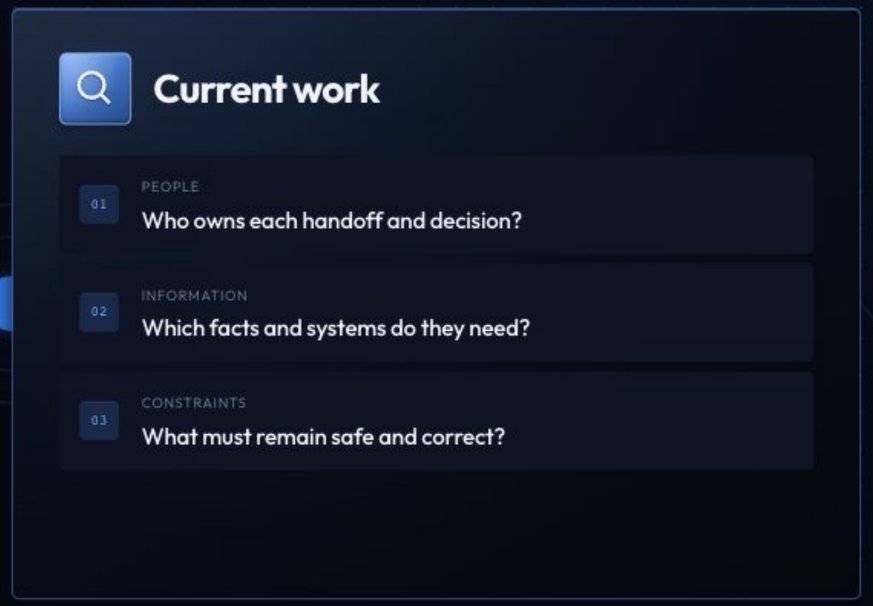
The unit of progress is a better business outcome—not another AI tool.

Redesign the work.
Then automate it.

The destination is clear. Now turn one valuable workflow into a repeatable result.

Map the work people actually do.

Identify the people, information, decisions and constraints behind the current process.



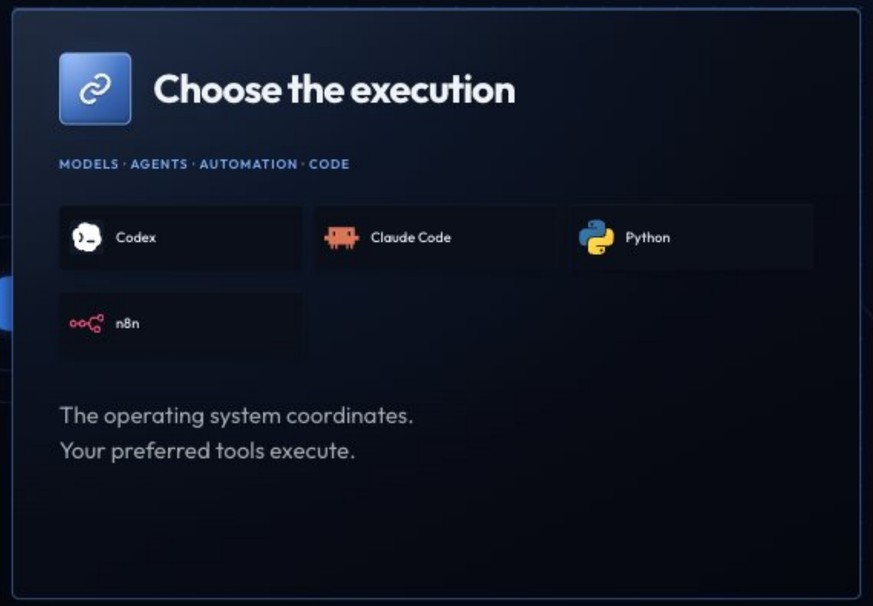
A faster bad process is still bad.

Remove waste, clarify decisions and preserve human judgment before you automate.



Choose the *right* way to execute.

Use skills, agents, code, workflow tools, MCP connections, APIs or people where each is most effective.



Earn the right to replace it.

Run the new workflow beside the current process and prove quality on comparable work.



From manual handoffs to measured results.

A customer support request becomes a connected workflow with checks, escalation and feedback.



Give the transformation a *management system.*

A repeatable process needs a decision loop, clear ownership and a team to deliver it.

Goal. Set the target.

2× output at half the operating cost, one workflow at a time.



Observe. Follow the real work.

Gather facts, traces and feedback to see where time, money and quality are lost.



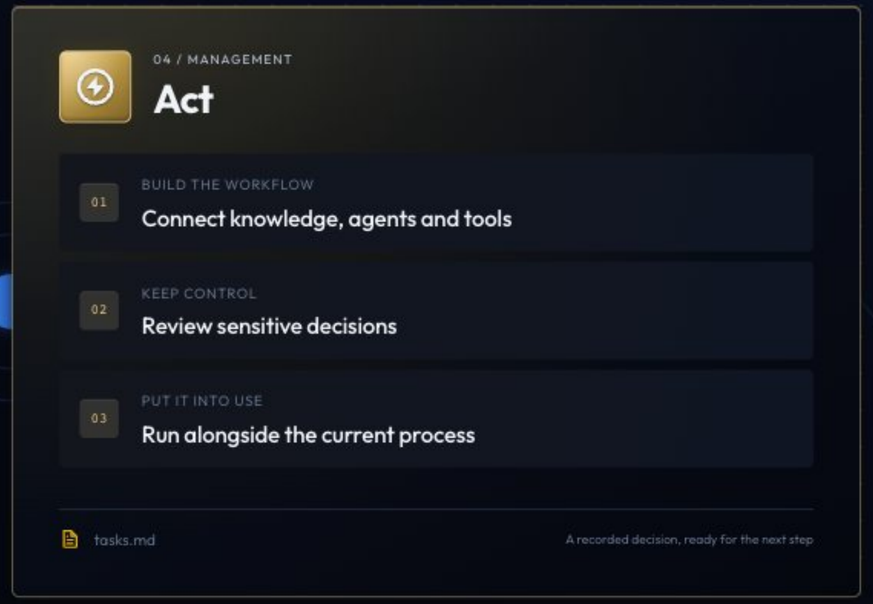
Decide. Choose the next change.

Use the evidence to prioritize an improvement and record the tradeoffs.



Act. Put the change to work.

Execute through the harness, workflows and tools, with people owning consequential decisions.



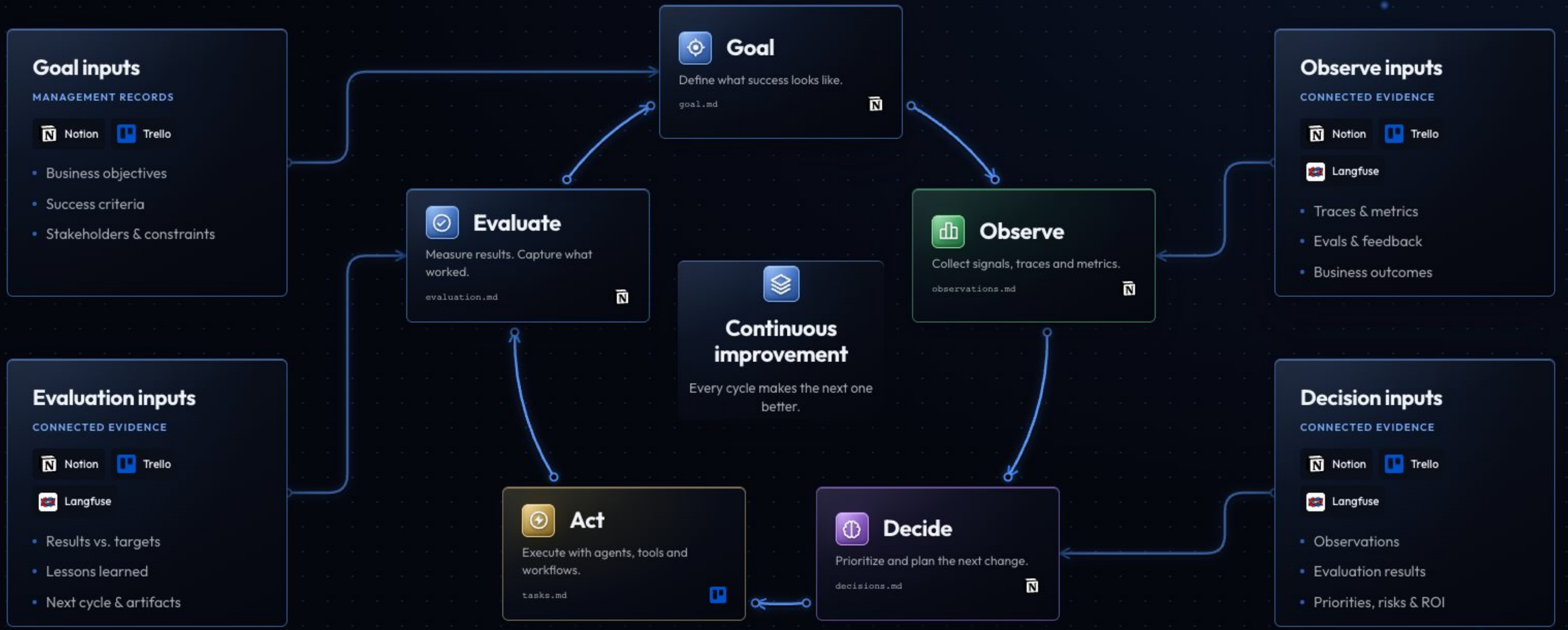
Evaluate.
Did it work?

Compare results with the goal, keep the lessons and feed the next cycle.



The management framework.

Goal → Observe → Decide → Act → Evaluate sits above the harness that executes the work.



The transformation *team.*

The management loop sets direction. Specialists turn each decision into working systems.

There is a *manager for the transformation.*

Business analysis, agent engineering and observability connect priorities, system design and measurable outcomes.



Understand before automating.

Process analysts uncover how the work really happens and where a better design will matter.



Design for the *whole system.*

Architecture makes context, models and tools work together toward a business result.



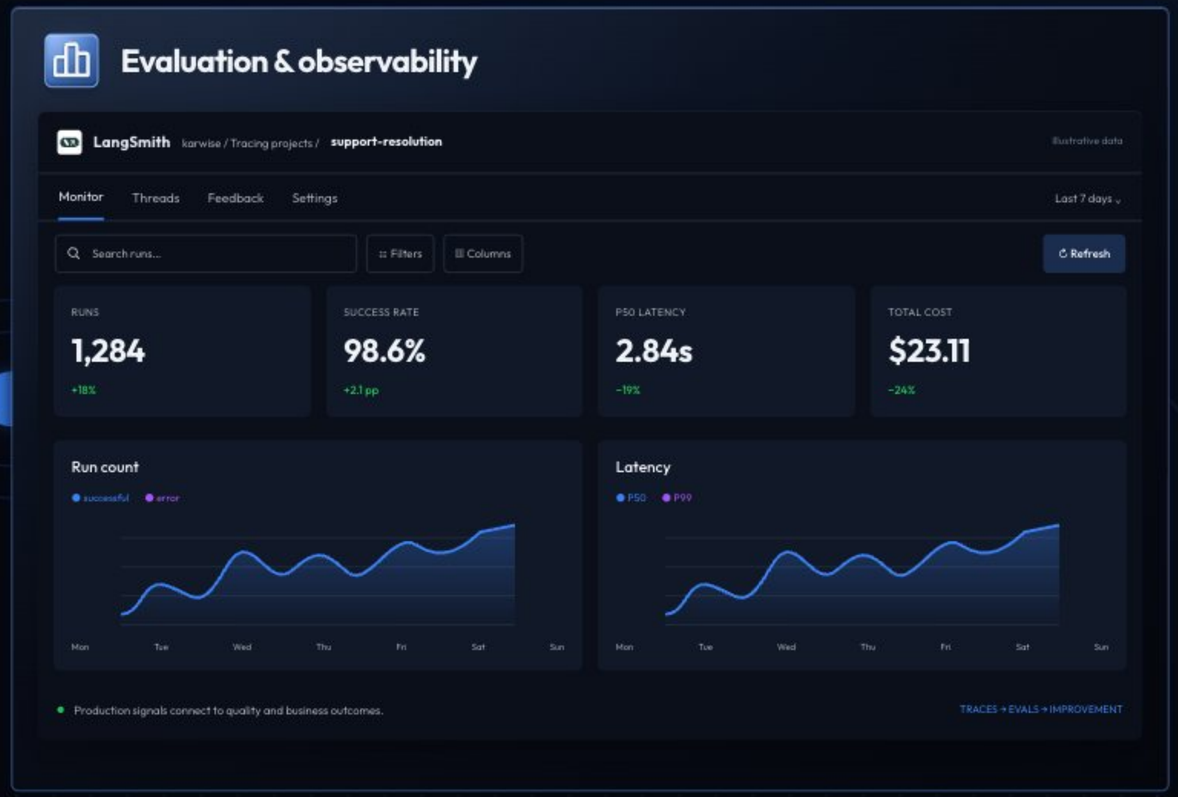
Turn a better idea into *daily work.*

Engineers build the agents, workflows and integrations that people can depend on.



Build Observability and Evals in.

The Evals engineer connects run traces, quality checks and business results from the start.



One transformation team. Every department.

Business teams own their outcomes. The transformation team helps them continuously improve.

TRANSFORMATION LEADERSHIP

Manager

Set priorities
Business value first

Connect the system
One shared architecture

Measure outcomes
Evidence for every decision

One team. Four complementary disciplines.

01 / Understand the work

Business analyst

Request → Resolve → Outcome

Map handoffs. Find the bottleneck.

Process map · opportunity brief

Notion · Trello

02 / Design the system

AI architect

- Company context
- Harness + memory
- Tools + guardrails

Architecture · model & tool choices

ChatGPT · Claude Code

03 / Build the capability

Agent engineer

```
01  workflow.run(request)
02  await agent.execute()
03  assert quality.passed
```

✓ build & checks passed

Agents · workflows · integrations

Codex · n8n · Python

04 / Prove the outcome

Evals engineer

Correctness	96%
Policy	100%
Task success	93%

Quality gates · traces · business impact

LangSmith · Langfuse

BUSINESS TEAMS OWN THEIR OUTCOMES

Shared capability → every department

Sales
Lead → opportunity
Salesforce

Support
Request → resolution
Zendesk

Marketing
Brief → campaign
Claude Code

Finance
Invoice → approval
Python

Operations
Work → delivery
n8n

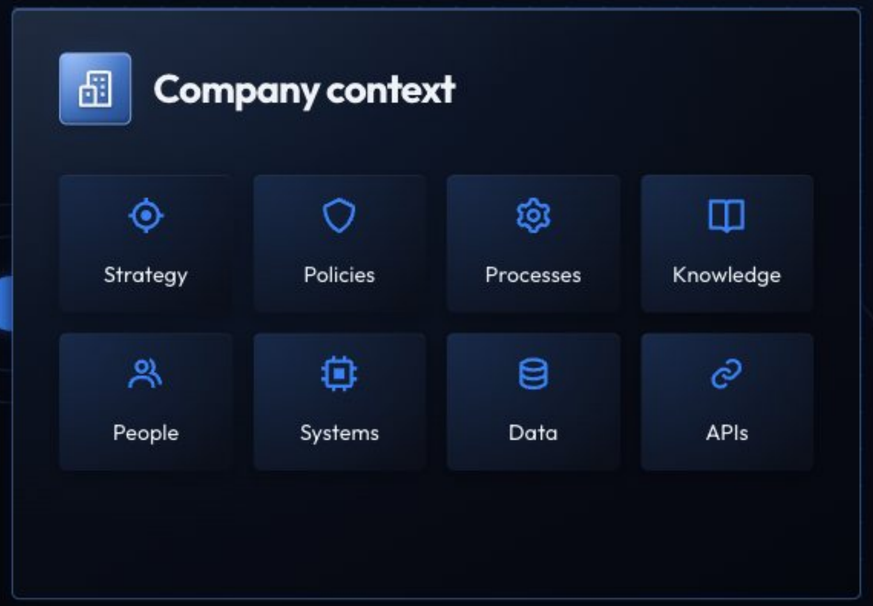
HR
Join → onboard
Notion

Build the shared company OS.

The team needs one foundation for context, execution, memory and evidence.

Your business is the context.

AI needs your strategy, policies, knowledge, people and systems to make useful decisions.



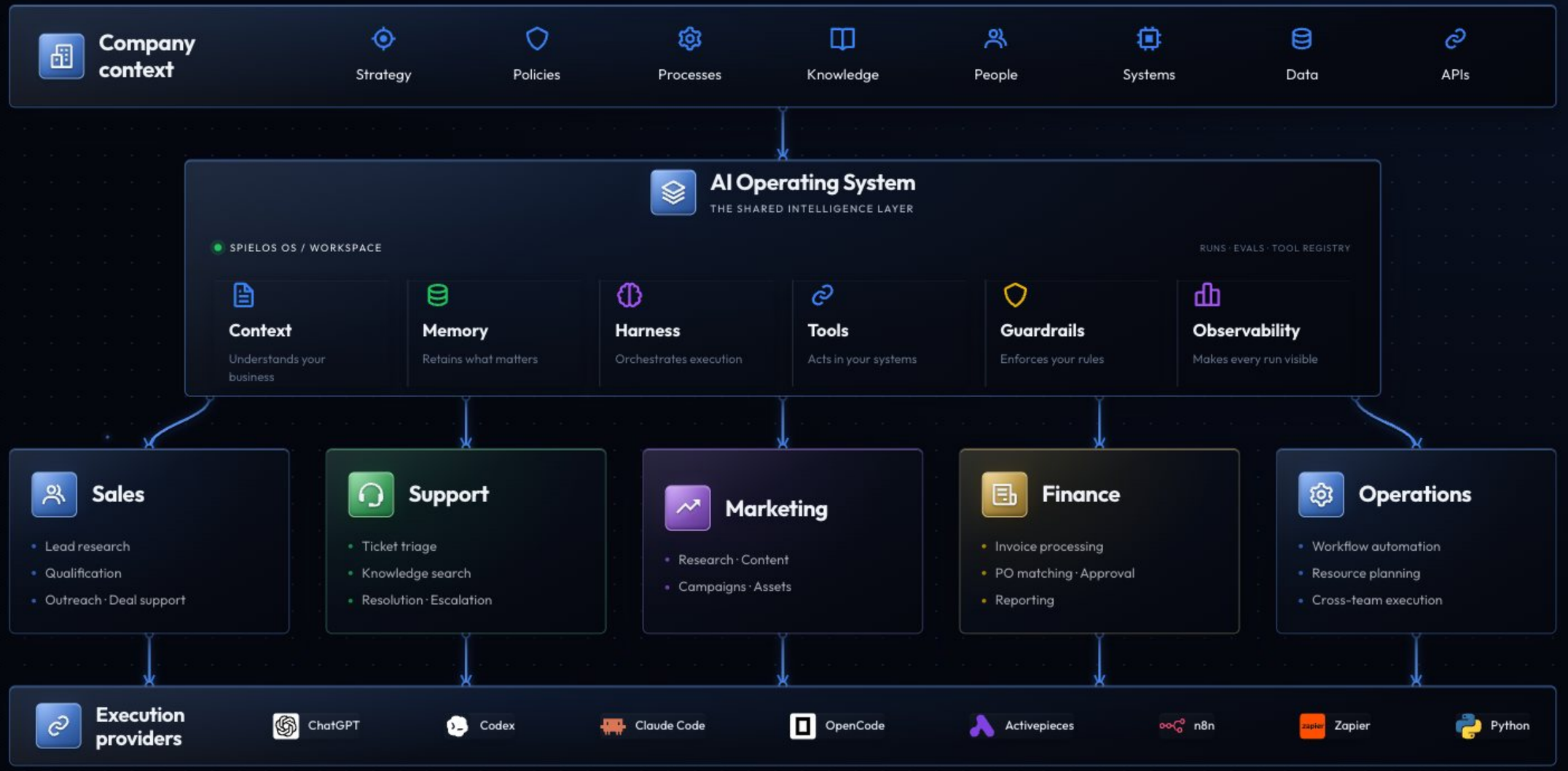
karwise is the shared operating system.

The product brings context, memory, the harness, tools, guardrails and observability into one workspace.



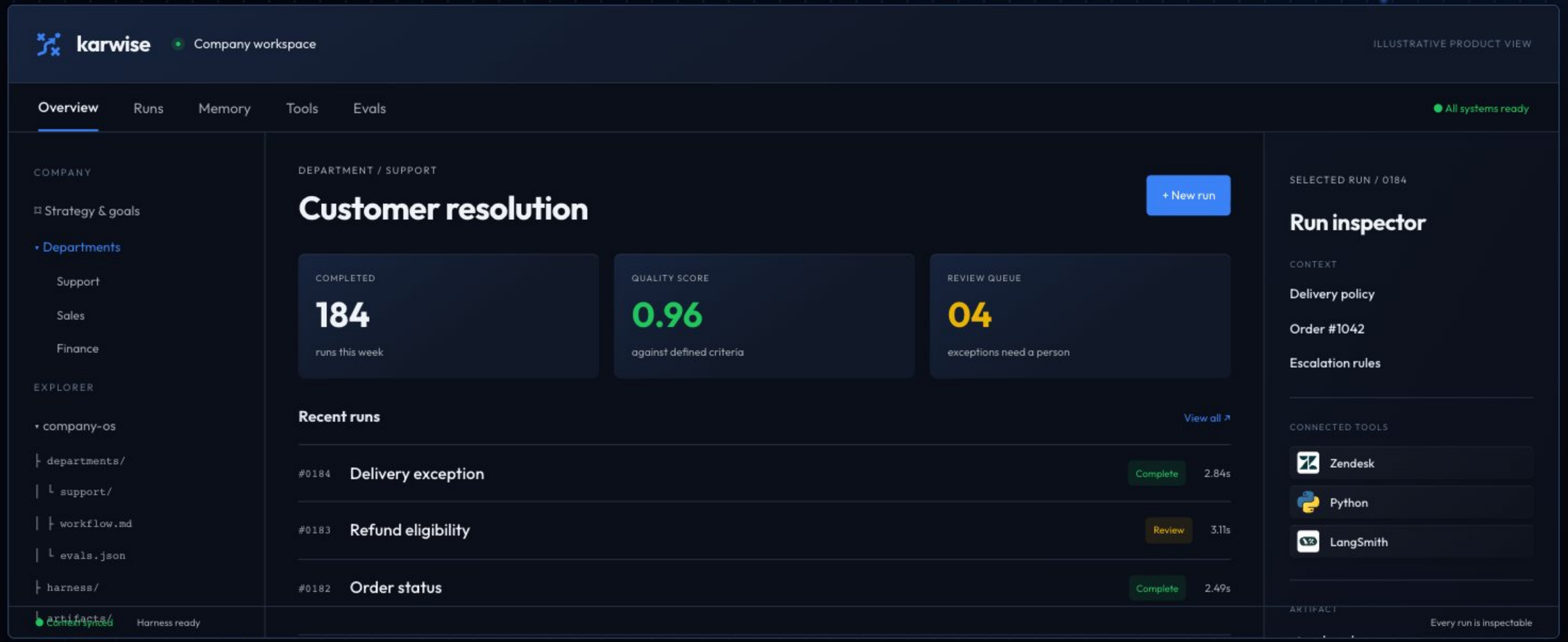
Company context. karwise. Department systems.

One AI operating system connects your knowledge, harness and execution providers to business outcomes.



The OS becomes a product surface.

Context, execution and evidence meet in one inspectable workspace.

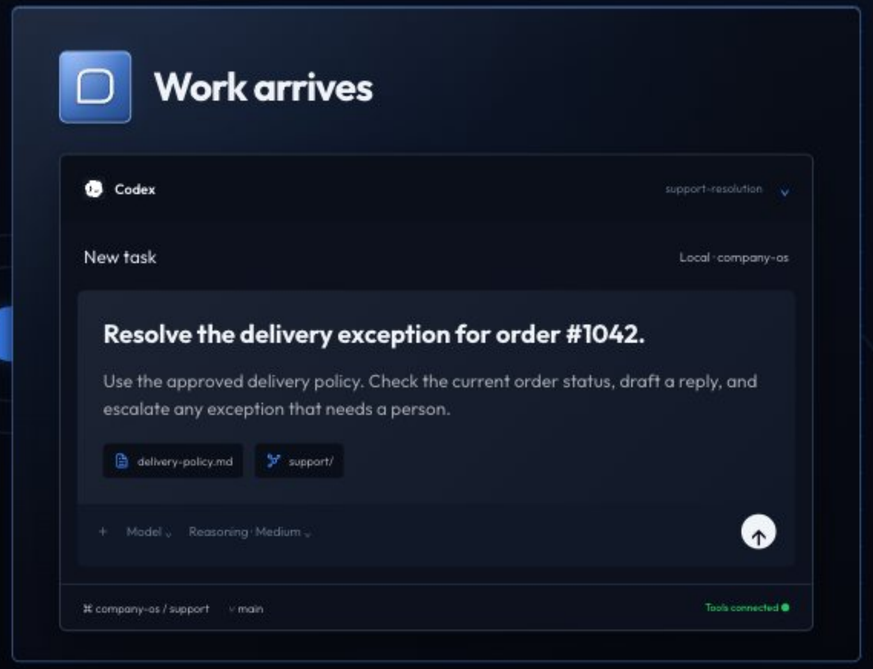


Inside the AI harness.

See how work runs, then use the same harness to build a workflow through MCP.

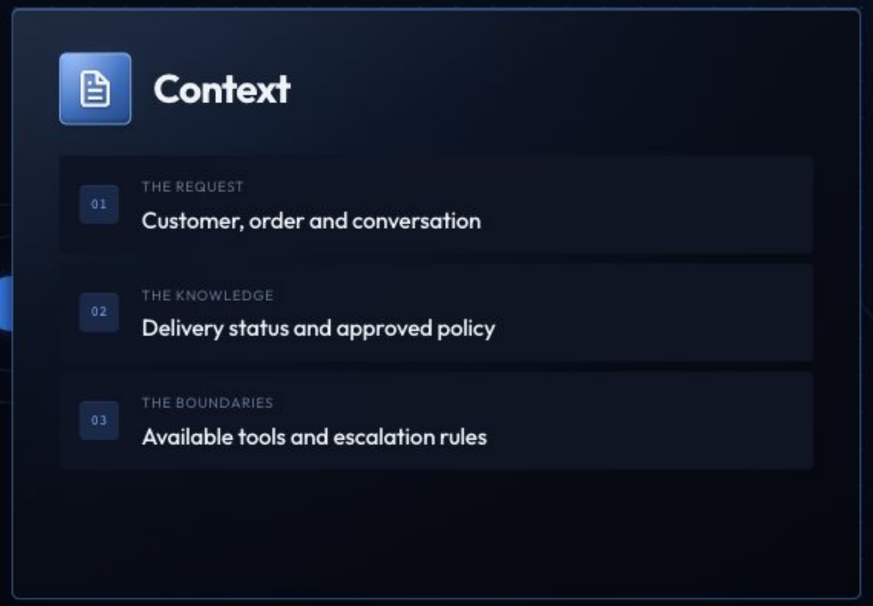
Every run starts with a *prompt.*

A prompt, task, trigger or event gives the harness a clear piece of work.



Brief the system *before it acts.*

Load the relevant request, knowledge, tools and boundaries for this task.



The harness runs the agent loop.

The runtime reasons, calls tools and observes results until the task meets its completion criteria.



Give specialists the *right work.*

Delegate research, building, analysis and review when a dedicated worker can improve the result.



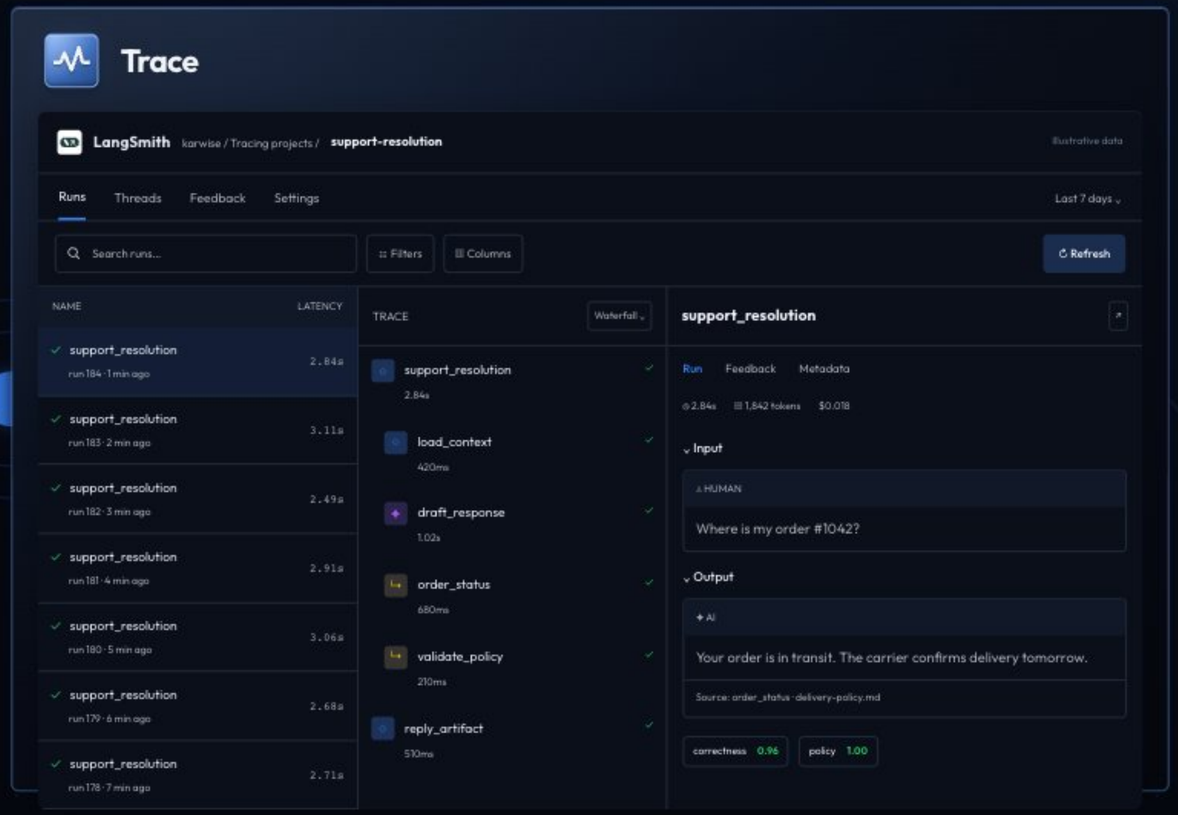
Completion has a quality bar.

Check correctness, policy and required work before accepting the result.



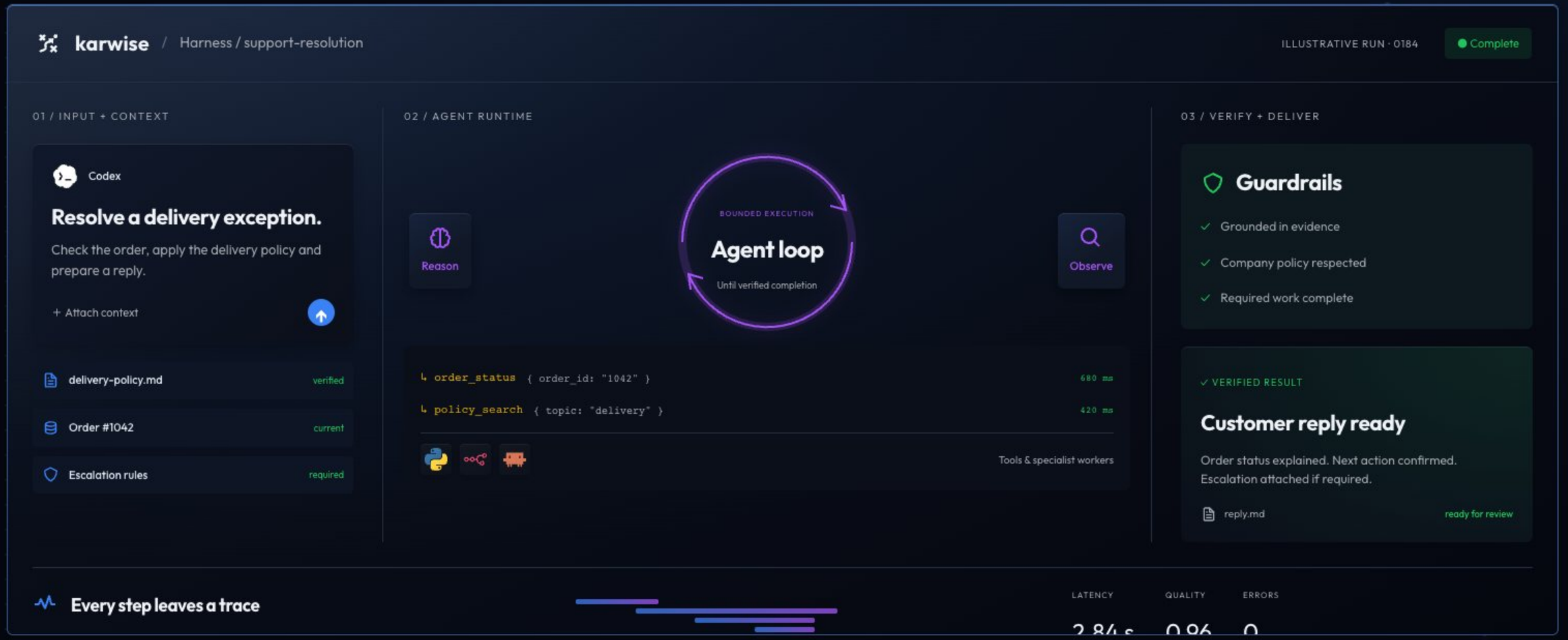
Every action leaves *evidence.*

Keep a trace of context, model calls, tools, timing, costs, errors and produced artifacts.



The harness turns prompts into verified work.

Context, runtime, tools, workers and guardrails form one observable execution loop.



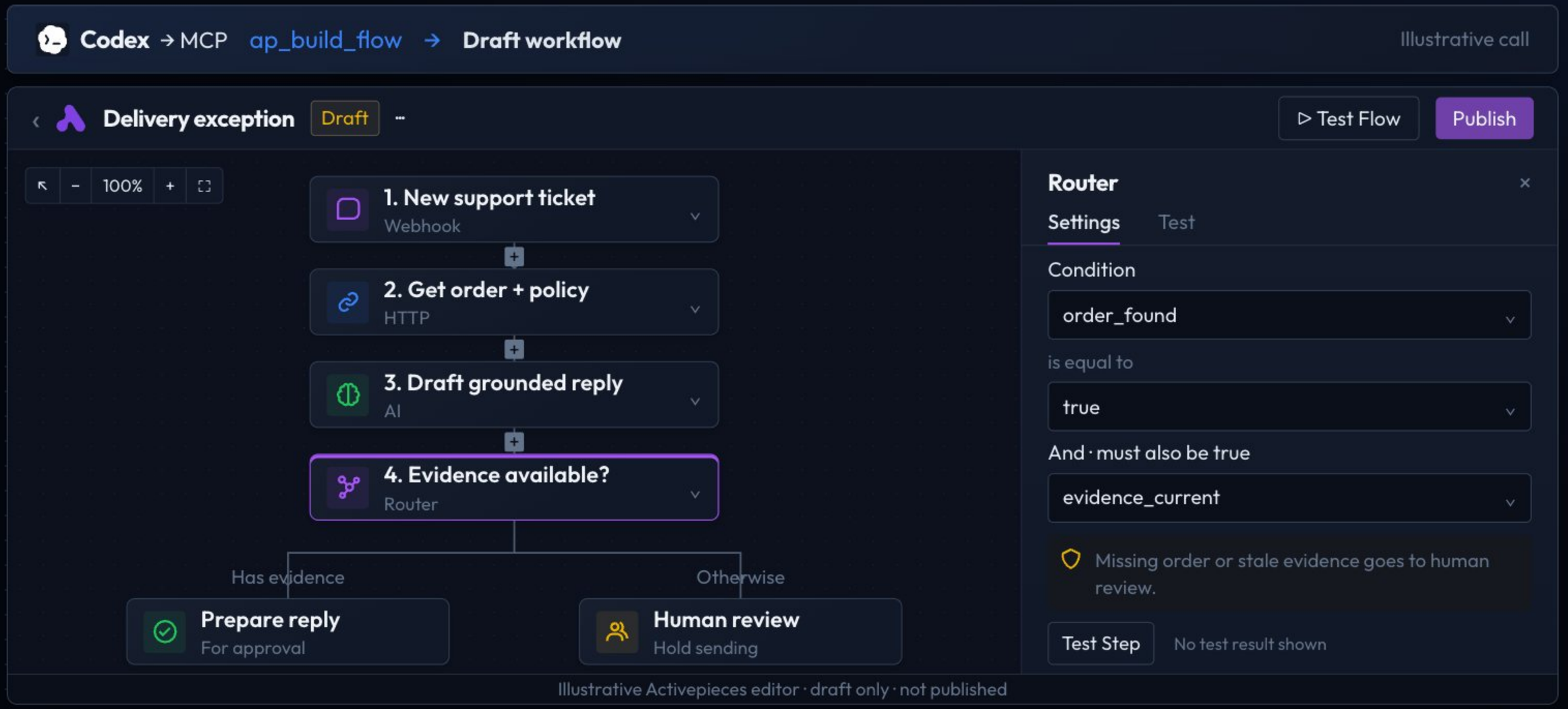
The build begins with an executable brief.

Request a delivery-exception workflow, attach the policy and acceptance tests, and keep deployment disabled until review.



Tool calls turn the plan into a workflow.

Use documented discovery and build tools to create a flow, then inspect its structure; this walkthrough is an illustrative example.



Tests check behavior *before release.*

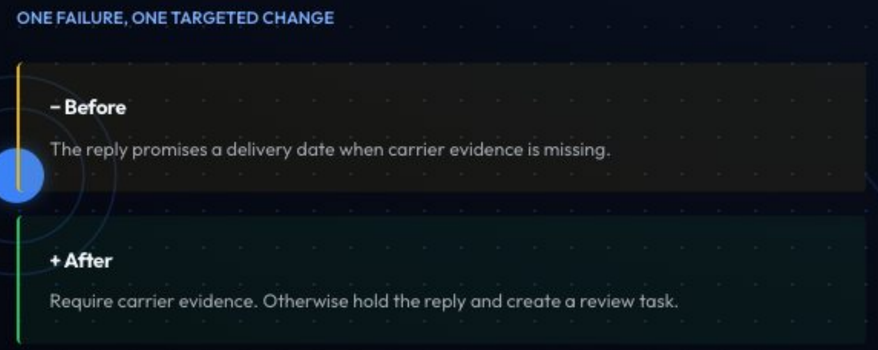
Validate the flow structure, then test representative cases against explicit expected outcomes; a valid flow is not proof of a good answer.

Release blocked

Acceptance case	Expected behavior	Example result
Known delivery status	Grounded reply	Pass
Missing carrier evidence	Hold + escalate	Fail
Refund request	Human approval	Pass
Policy exception	Owner review	Pass

A failed case shows what needs to change.

An unknown order must not produce an invented delivery date; repair the branch and rerun the failing case plus regression tests.

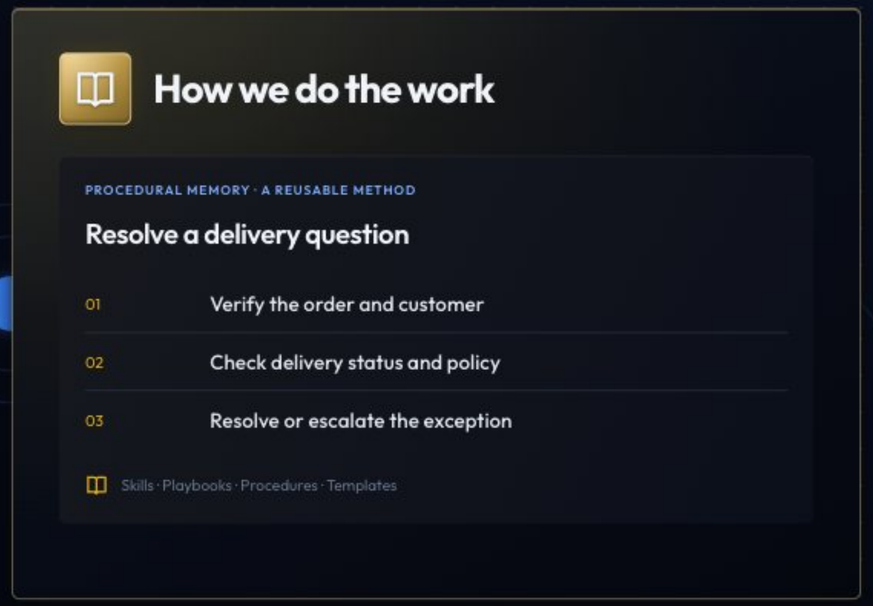


Make the next run *better informed.*

Execution produces evidence. Memory keeps the knowledge and methods worth reusing.

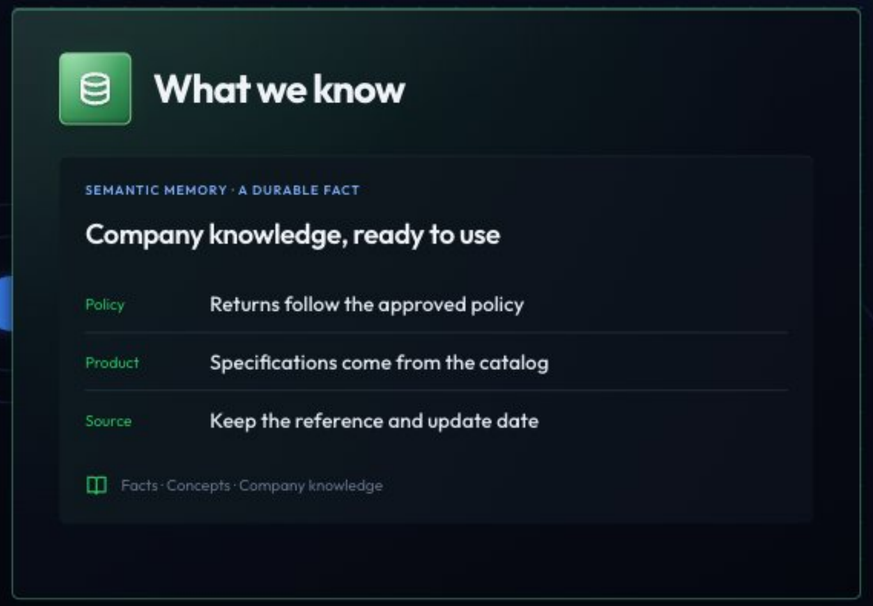
Procedural memory knows *how*.

Procedural memory stores proven methods so the next run can reuse them.



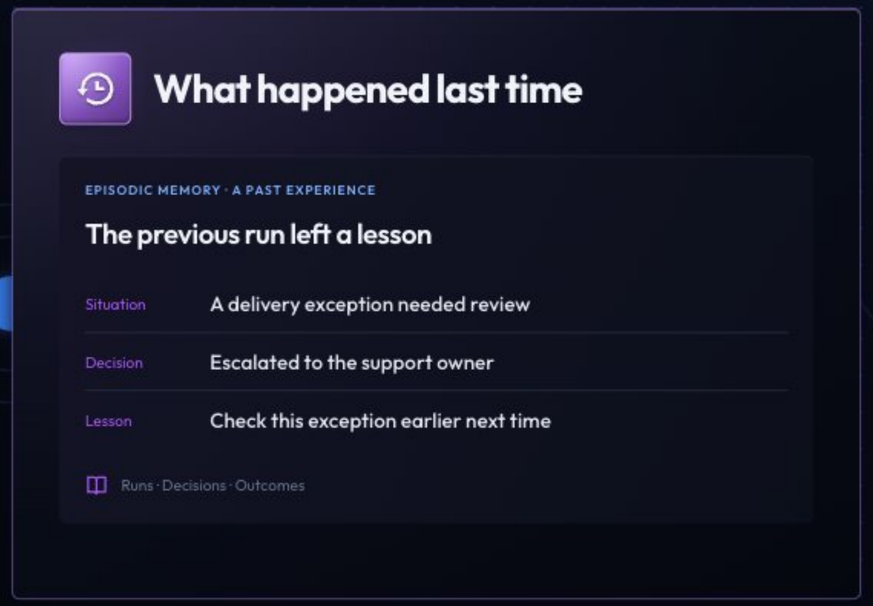
Semantic memory knows *what*.

Semantic memory keeps useful facts and knowledge available with their sources.



Episodic memory remembers what happened.

Episodic memory captures past situations, decisions and outcomes worth learning from.



Give this task the *right* knowledge.

Retrieve what matters, leave out what does not and assemble a focused working context.



The next run starts better informed.

Methods, facts and experience feed the work; the result adds another useful lesson.

01 / COMPANY MEMORY

PROCEDURAL

How to do it

Delivery exception playbook

Procedure · approved v3

SEMANTIC

What is true

Delivery policy & product facts

Knowledge · source linked

EPISODIC

What happened

Resolved support run / 0183

Experience · reviewed lesson

02 / CONTEXT BUILDER

Retrieve. Select. Compose.

Resolve delivery exception #1042

Delivery exception playbook METHOD	0.98
Approved delivery policy FACT	0.96
Previous resolved exception LESSON	0.91

Working context — 2,480 tokens

Only the relevant sources, with their boundaries.

COMPOSED BRIEF → HARNESS

Request + facts + method + useful history

03 / APPLY + LEARN

Run with context

Reason → Act → Observe

Execute through the harness.
Verify the business result.

EPISODE / 0184

A useful lesson, retained.

Capture the decision, its outcome and what should improve next time.

✓ Reviewed before reuse

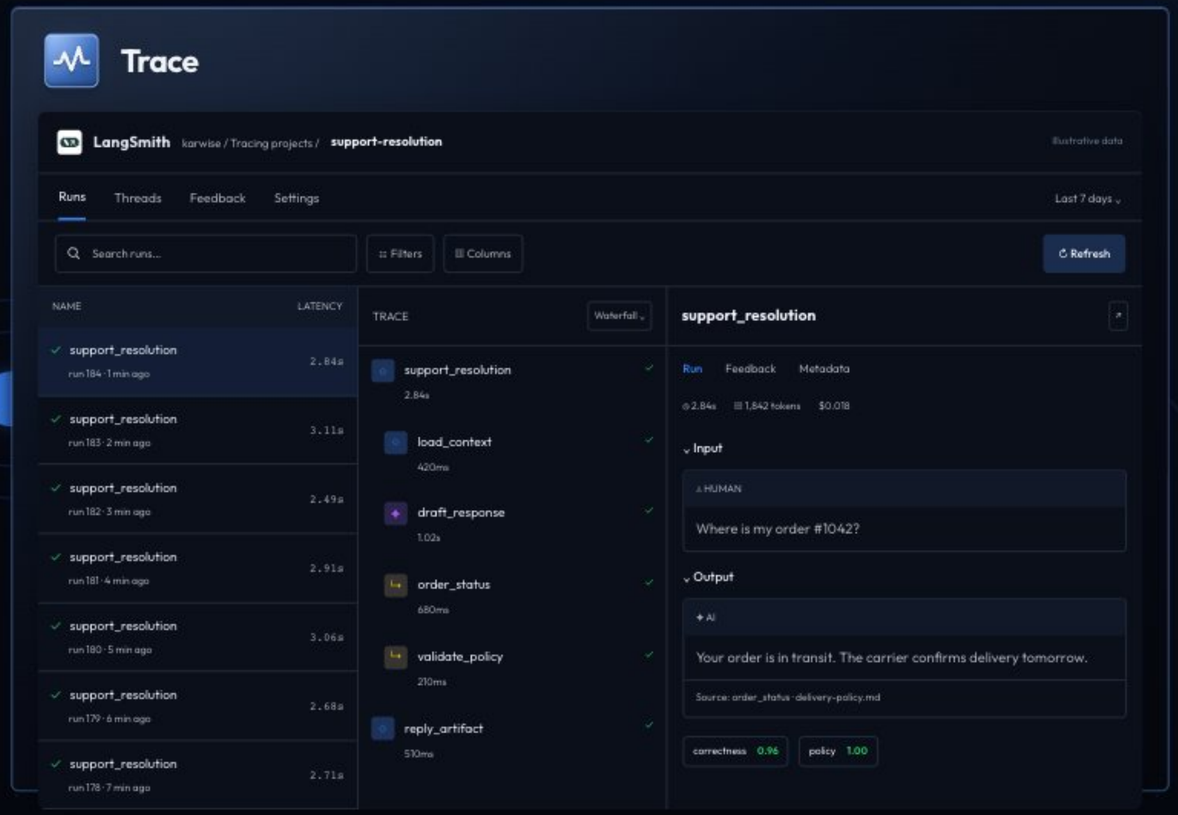
Review the outcome → update the method or knowledge → improve the next run

Prove the result. Earn the right to scale.

Trace what happened, evaluate quality and measure the business outcome before expanding.

The trace explains what happened.

Follow the run from its initial context through every action to its final artifact.



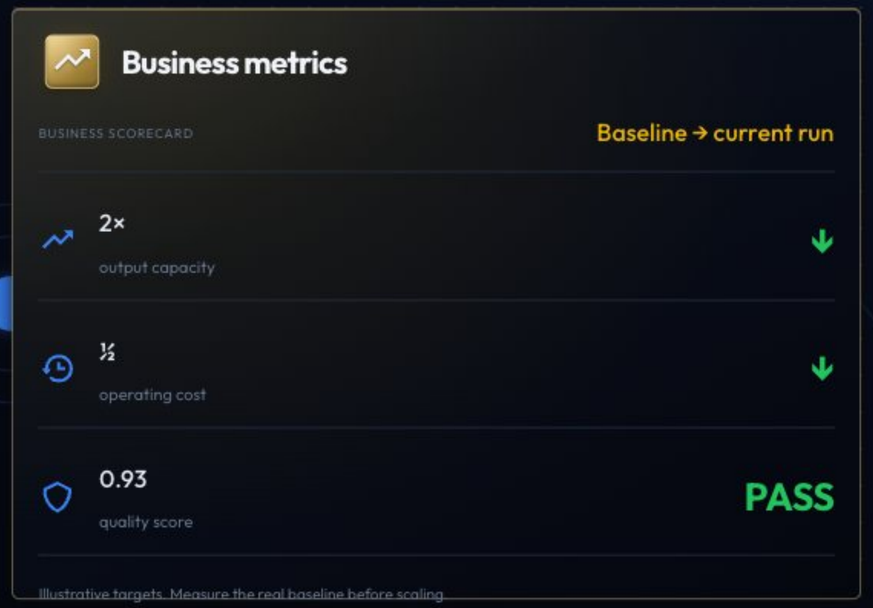
Evals tell us: was it good?

Test correctness, relevance, policy and task success against explicit thresholds.



The scorecard asks: did it matter?

Compare time, cost, quality and adoption with the business outcome you set out to improve.



Trace → Evaluate → Learn → Improve.

Observability feeds Observe; Evals and business outcomes feed Evaluate.



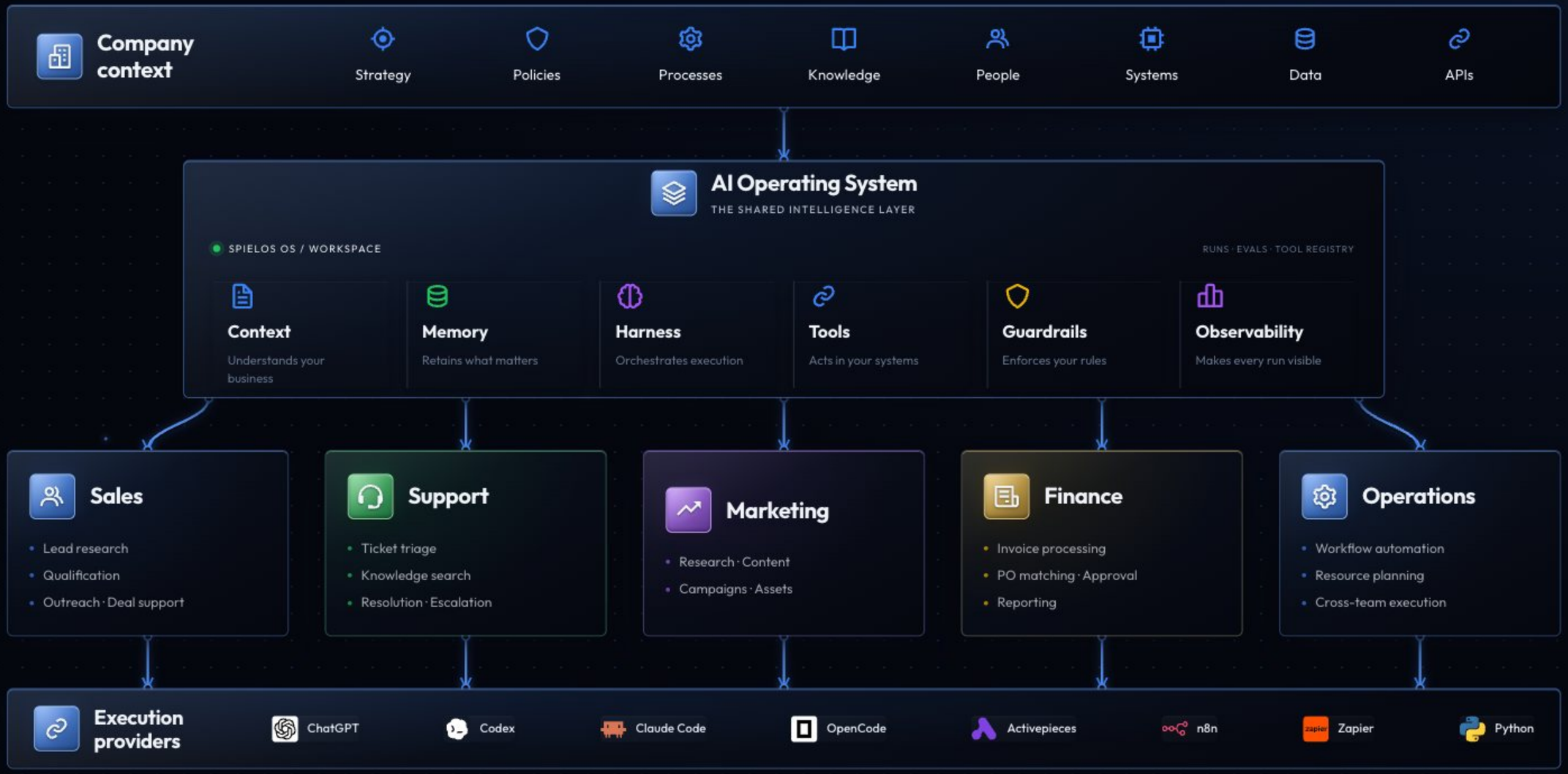
Scale what you *can prove.*

Expand a successful method into new workflows and departments, one measured step at a time.



Specialized teams. Shared intelligence.

Each department moves toward its own outcomes on one company-wide foundation.



karwise

Thank you for your time.